Lab 7 — Fuzz Testing with Hypothesis

Cloud Storage Tier | Repository L6_L7_12

Name: Sanjith Suresh Kumar
SRN / ID: PES1UG24AM250
Subject: Software Engineering Laboratory

1. Objective

Explore fuzz testing as a security and quality-assurance technique by identifying and fixing input-validation bugs in a Python script (processor.py) using the Hypothesis fuzzing library.

2. Learning Outcomes

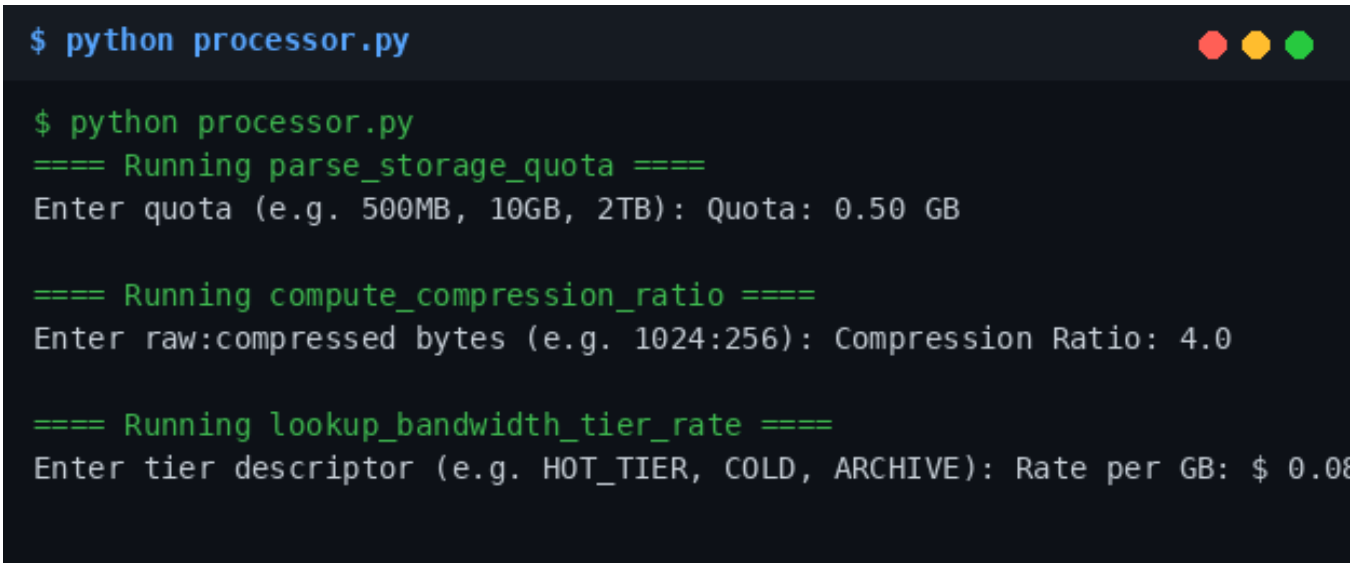
- Understand the purpose and methodology of fuzz testing.
- Apply property-based fuzzing using Hypothesis.
- Detect and analyse bugs caused by edge-case inputs.
- Improve code robustness through defensive programming.

3. Tools & Setup

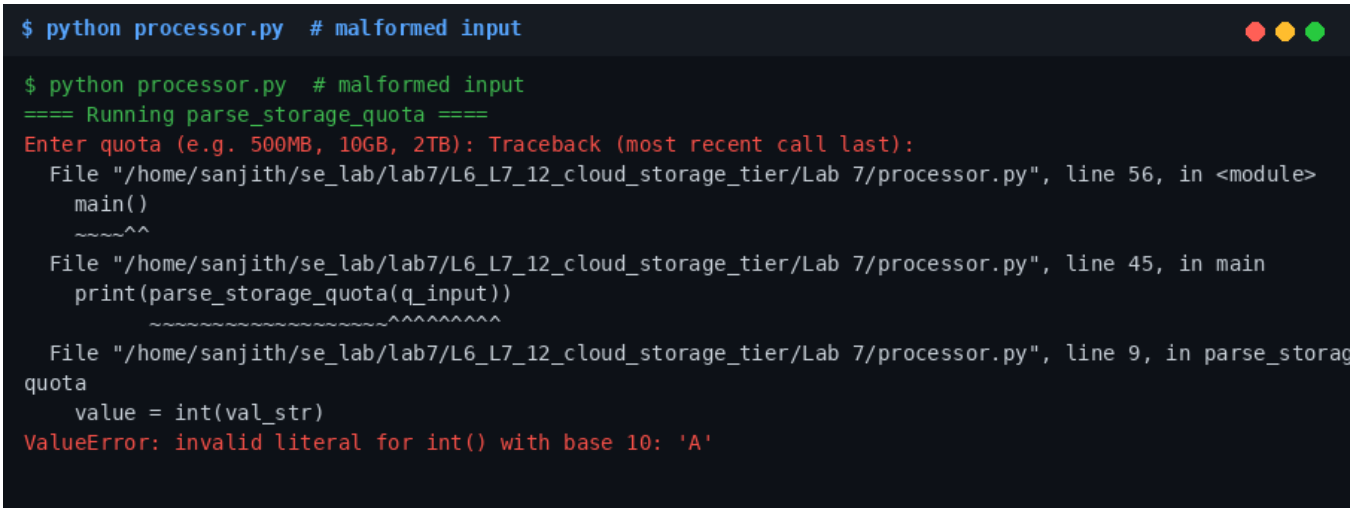
Python 3.13.13 | pytest 9.2.0 | hypothesis 6.170.15
Installed with: pip install hypothesis pytest

Step 2 — Run the buggy application (processor.py)

Running the original, unmodified processor.py with normal inputs. The module parses storage quotas, computes compression ratios, and looks up bandwidth tier rates.



However, feeding the same program a malformed input (a non-numeric quota) causes it to crash with an unhandled ValueError — exactly the class of bug fuzzing is designed to find automatically:



Step 4 — Run the completed fuzz tests (they fail)

test_processor.py was completed with property-based Hypothesis tests. Each test feeds its function arbitrary text or None and asserts it never crashes. On the buggy code all three tests fail, and Hypothesis automatically shrinks each failure to its smallest counter-example (e.g. an empty string, None, a missing colon, or division by zero).

```
$ python -m pytest test_processor.py -v
```

```
$ python -m pytest test_processor.py -v
============================= test session starts ==============================
platform linux -- Python 3.13.13, pytest-9.2.0, pluggy-1.6.0 -- /home/sanjith/miniconda3/bin/python3
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: /home/sanjith/se_lab/lab7
plugins: hypothesis-6.170.15, anyio-4.12.1
collecting ... collected 3 items

test_processor.py::test_parse_storage_quota_no_crash FAILED              [ 33%]
test_processor.py::test_compute_compression_ratio_safe FAILED            [ 66%]
test_processor.py::test_lookup_bandwidth_tier_rate_safe FAILED           [100%]

================================== FAILURES ===================================
_____________________ test_parse_storage_quota_no_crash _____________________
  + Exception Group Traceback (most recent call last):
  |   File "/home/sanjith/se_lab/lab7/test_processor.py", line 23, in test_parse_storage_quota_no_crash
  |     def test_parse_storage_quota_no_crash(s):
  |                    ^^^
  | ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "/home/sanjith/se_lab/lab7/test_processor.py", line 26, in test_parse_storage_quota_no_crash
    |     parse_storage_quota(s)
    |     ~~~~~~~~~~~~~~~~~~~^^^
    |   File "/home/sanjith/se_lab/lab7/processor.py", line 9, in parse_storage_quota
    |     value = int(val_str)
    | ValueError: invalid literal for int() with base 10: ''
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "/home/sanjith/se_lab/lab7/test_processor.py", line 28, in test_parse_storage_quota_no_crash
    |     assert False, f"parse_storage_quota crashed with {s!r}: {exc}"
    | AssertionError: parse_storage_quota crashed with '': invalid literal for int() with base 10: ''
    | assert False
    | Failing test case: test_parse_storage_quota_no_crash(
    |     s='',
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "/home/sanjith/se_lab/lab7/test_processor.py", line 26, in test_parse_storage_quota_no_crash
    |     parse_storage_quota(s)
    |     ~~~~~~~~~~~~~~~~~~~^^^
    |   File "/home/sanjith/se_lab/lab7/processor.py", line 6, in parse_storage_quota
    |     cleaned = quota_str.strip().upper()
    |               ^^^^^^^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'strip'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "/home/sanjith/se_lab/lab7/test_processor.py", line 28, in test_parse_storage_quota_no_crash
    |     assert False, f"parse_storage_quota crashed with {s!r}: {exc}"
    | AssertionError: parse_storage_quota crashed with None: 'NoneType' object has no attribute 'strip'
    | assert False
    | Failing test case: test_parse_storage_quota_no_crash(
    |     s=None,
    | )
    +------------------------------------
```

```
$ python -m pytest test_processor.py -v
```

```
$ python -m pytest test_processor.py -v
============================= test session starts ==============================
platform linux -- Python 3.13.13, pytest-9.2.0, pluggy-1.6.0 -- /home/sanjith/miniconda3/bin/python3
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: /home/sanjith/se_lab/lab7
plugins: hypothesis-6.170.15, anyio-4.12.1
collecting ... collected 3 items

test_processor.py::test_parse_storage_quota_no_crash FAILED              [ 33%]
test_processor.py::test_compute_compression_ratio_safe FAILED            [ 66%]
test_processor.py::test_lookup_bandwidth_tier_rate_safe FAILED           [100%]

================================== FAILURES ===================================
_____________________ test_compute_compression_ratio_safe _____________________
  + Exception Group Traceback (most recent call last):
  |   File "/home/sanjith/se_lab/lab7/test_processor.py", line 32, in test_compute_compression_ratio_safe
  |     def test_compute_compression_ratio_safe(s):
  |                    ^^^
  | ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "/home/sanjith/se_lab/lab7/test_processor.py", line 35, in test_compute_compression_ratio_safe
    |     compute_compression_ratio(s)
    |     ~~~~~~~~~~~~~~~~~~~~~~~~~^^^
    |   File "/home/sanjith/se_lab/lab7/processor.py", line 24, in compute_compression_ratio
    |     raw_bytes = float(raw_str)
    | ValueError: could not convert string to float: ''
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "/home/sanjith/se_lab/lab7/test_processor.py", line 37, in test_compute_compression_ratio_safe
    |     assert False, f"compute_compression_ratio crashed with {s!r}: {exc}"
    | AssertionError: compute_compression_ratio crashed with ':': could not convert string to float: ''
    | assert False
    | Failing test case: test_compute_compression_ratio_safe(
    |     s=':',
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "/home/sanjith/se_lab/lab7/test_processor.py", line 35, in test_compute_compression_ratio_safe
    |     compute_compression_ratio(s)
    |     ~~~~~~~~~~~~~~~~~~~~~~~~~^^^
    |   File "/home/sanjith/se_lab/lab7/processor.py", line 23, in compute_compression_ratio
    |     raw_str, comp_str = size_metric_str.split(':')
    |     ^^^^^^^^^^^^^^^^
    | ValueError: not enough values to unpack (expected 2, got 1)
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "/home/sanjith/se_lab/lab7/test_processor.py", line 37, in test_compute_compression_ratio_safe
    |     assert False, f"compute_compression_ratio crashed with {s!r}: {exc}"
    | AssertionError: compute_compression_ratio crashed with '': not enough values to unpack (expected 2, got 1)
    | assert False
    | Failing test case: test_compute_compression_ratio_safe(
    |     s='',
    | )
    +---------------- 3 ----------------
    | Traceback (most recent call last):
    |   File "/home/sanjith/se_lab/lab7/test_processor.py", line 35, in test_compute_compression_ratio_safe
    |     compute_compression_ratio(s)
    |     ~~~~~~~~~~~~~~~~~~~~~~~~~^^^
    |   File "/home/sanjith/se_lab/lab7/processor.py", line 23, in compute_compression_ratio
    |     raw_str, comp_str = size_metric_str.split(':')
    |                         ^^^^^^^^^^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'split'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "/home/sanjith/se_lab/lab7/test_processor.py", line 37, in test_compute_compression_ratio_safe
    |     assert False, f"compute_compression_ratio crashed with {s!r}: {exc}"
    | AssertionError: compute_compression_ratio crashed with None: 'NoneType' object has no attribute 'split'
    | assert False
    | Failing test case: test_compute_compression_ratio_safe(
    |     s=None,
    | )
    +------------------------------------
```

```
$ python -m pytest test_processor.py -v
```

```
$ python -m pytest test_processor.py -v
============================= test session starts ==============================
platform linux -- Python 3.13.13, pytest-9.2.0, pluggy-1.6.0 -- /home/sanjith/miniconda3/bin/python3
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: /home/sanjith/se_lab/lab7
plugins: hypothesis-6.170.15, anyio-4.12.1
collecting ... collected 3 items

test_processor.py::test_parse_storage_quota_no_crash FAILED              [ 33%]
test_processor.py::test_compute_compression_ratio_safe FAILED            [ 66%]
test_processor.py::test_lookup_bandwidth_tier_rate_safe FAILED           [100%]

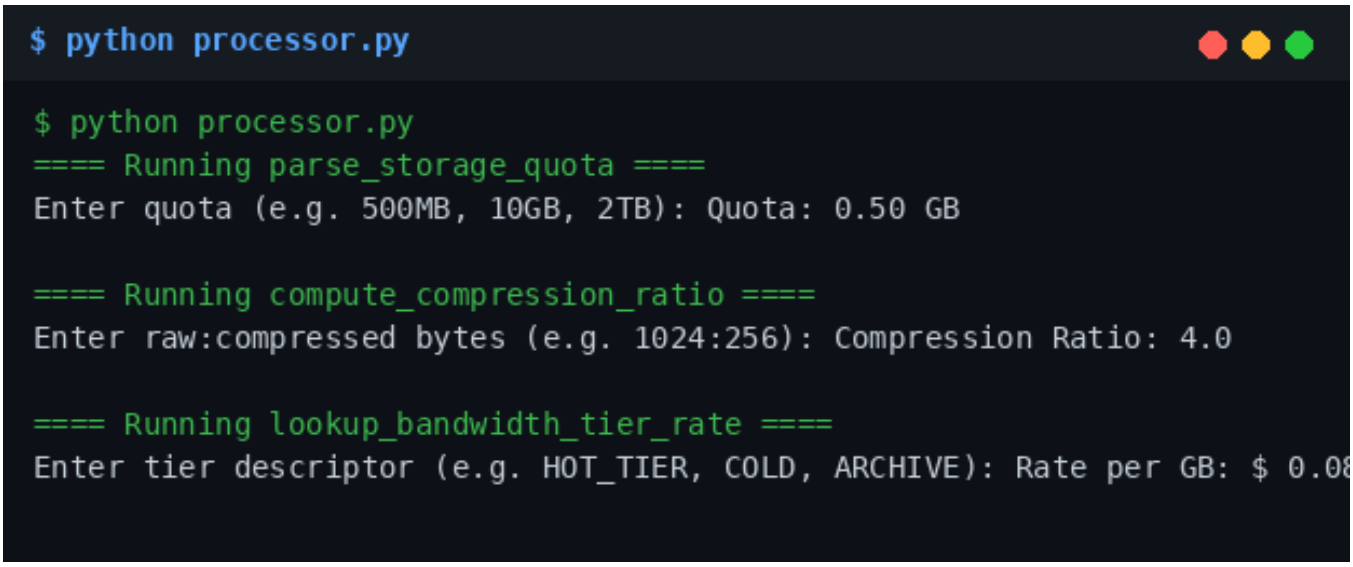
=================================== FAILURES ===================================
_____________________ test_lookup_bandwidth_tier_rate_safe _____________________
  + Exception Group Traceback (most recent call last):
  |   File "/home/sanjith/se_lab/lab7/test_processor.py", line 41, in test_lookup_bandwidth_tier_rate_safe
  |     def test_lookup_bandwidth_tier_rate_safe(s):
  |                    ^^^
  | ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "/home/sanjith/se_lab/lab7/test_processor.py", line 44, in test_lookup_bandwidth_tier_rate_safe
    |     lookup_bandwidth_tier_rate(s)
    |     ~~~~~~~~~~~~~~~~~~~~~~~~~~^^^
    |   File "/home/sanjith/se_lab/lab7/processor.py", line 40, in lookup_bandwidth_tier_rate
    |     return tier_rates[prefix]
    |            ~~~~~~~~~~^^^^^^^^
    | KeyError: ''
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "/home/sanjith/se_lab/lab7/test_processor.py", line 46, in test_lookup_bandwidth_tier_rate_safe
    |     assert False, f"lookup_bandwidth_tier_rate crashed with {s!r}: {exc}"
    | AssertionError: lookup_bandwidth_tier_rate crashed with '': ''
    | assert False
    | Failing test case: test_lookup_bandwidth_tier_rate_safe(
    |     s='',
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "/home/sanjith/se_lab/lab7/test_processor.py", line 44, in test_lookup_bandwidth_tier_rate_safe
    |     lookup_bandwidth_tier_rate(s)
    |     ~~~~~~~~~~~~~~~~~~~~~~~~~~^^^
    |   File "/home/sanjith/se_lab/lab7/processor.py", line 34, in lookup_bandwidth_tier_rate
    |     prefix = tier_descriptor.strip().upper()[:3]
    |              ^^^^^^^^^^^^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'strip'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "/home/sanjith/se_lab/lab7/test_processor.py", line 46, in test_lookup_bandwidth_tier_rate_safe
    |     assert False, f"lookup_bandwidth_tier_rate crashed with {s!r}: {exc}"
    | AssertionError: lookup_bandwidth_tier_rate crashed with None: 'NoneType' object has no attribute 'strip'
    | assert False
    | Failing test case: test_lookup_bandwidth_tier_rate_safe(
    |     s=None,
    | )
    +------------------------------------
=========================== short test summary info ============================
FAILED test_processor.py::test_parse_storage_quota_no_crash - ExceptionGroup:...
FAILED test_processor.py::test_compute_compression_ratio_safe - ExceptionGrou...
FAILED test_processor.py::test_lookup_bandwidth_tier_rate_safe - ExceptionGro...
============================== 3 failed in 0.15s ===============================

=========================== short test summary info ============================
FAILED test_processor.py::test_parse_storage_quota_no_crash - ExceptionGroup:...
FAILED test_processor.py::test_compute_compression_ratio_safe - ExceptionGrou...
FAILED test_processor.py::test_lookup_bandwidth_tier_rate_safe - ExceptionGro...
============================== 3 failed in 0.15s ===============================
```

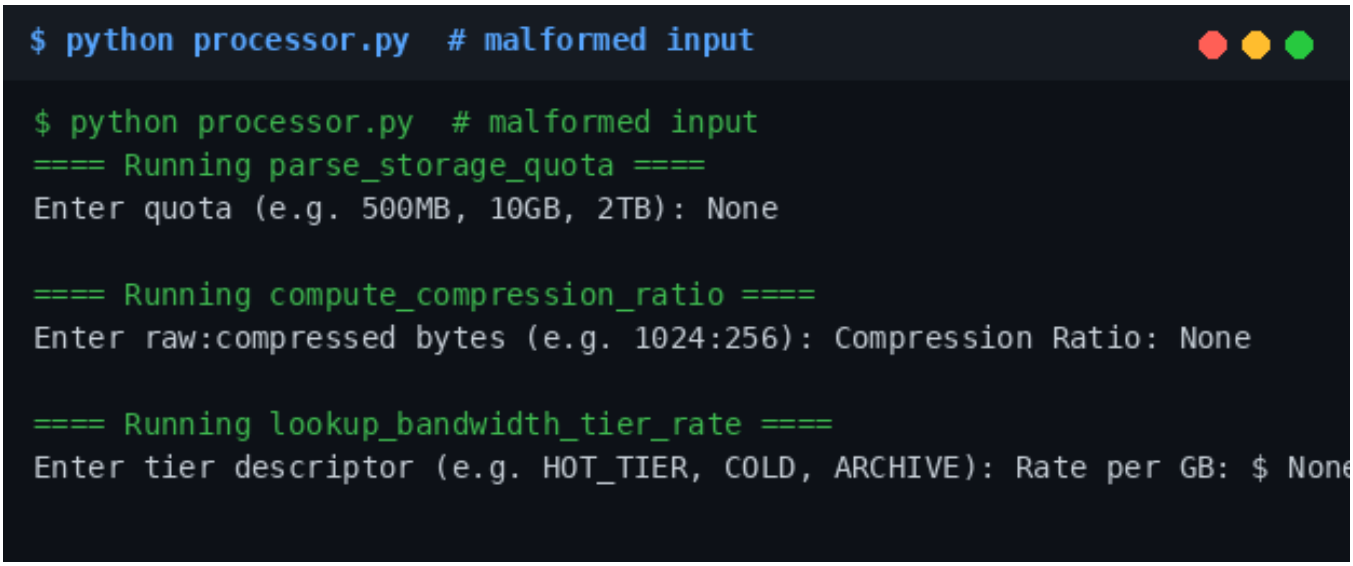
Step 5 — Fix processor.py (defensive programming)

Each function was hardened so that it validates its input and returns None instead of raising an unhandled exception:
- parse_storage_quota: regex validation of '<digits><unit>' plus None/type guards.
- compute_compression_ratio: guards for missing ':' , non-numeric values, and zero compressed bytes (division by zero).
- lookup_bandwidth_tier_rate: None/type guards and a dict .get() lookup that tolerates unknown prefixes.

Running the fixed processor.py with valid input produces the correct output:

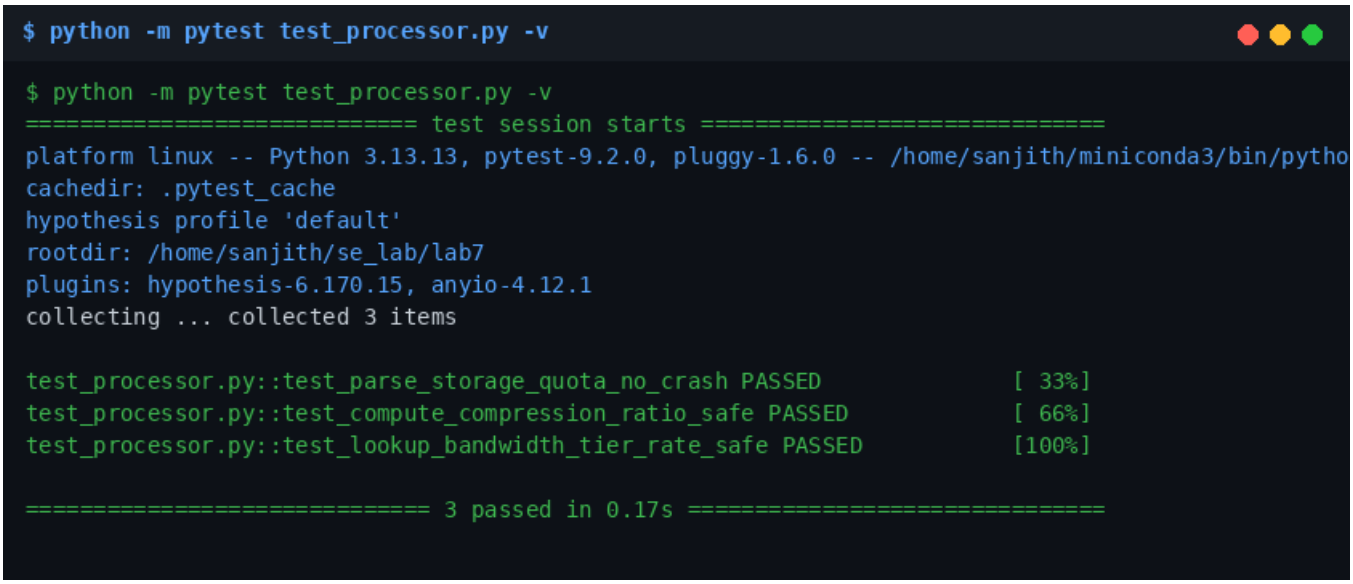


And with malformed input it no longer crashes — it degrades gracefully to None:



Step 6 — Re-run the fuzz tests (they pass)

After the fix, all three property-based tests pass, confirming the functions are now robust against the full range of unexpected inputs generated by Hypothesis.



4. Bugs found by fuzzing

parse_storage_quota

AttributeError on None (no .strip), ValueError on empty / non-numeric value, KeyError on unsupported unit.

compute_compression_ratio

AttributeError on None, ValueError when ':' is missing or values are non-numeric, ZeroDivisionError when compressed bytes = 0.

lookup_bandwidth_tier_rate

AttributeError on None, KeyError on empty or unknown tier prefix.

5. Conclusion

Hypothesis property-based fuzzing automatically uncovered multiple edge-case crashes that manual unit tests would likely have missed. Hardening the functions with input validation and graceful handling eliminated every crash; the final test run passes 3/3.